Practice Report

Delivery 01

Student: Phillip Mitchell Brule

Student: Abayev Nurzhan

Student: Arnau Ranchal Marín

Student: Ŝeyedehsara Yazdi Bahri

Escola Tècnica Superior
d'Enginyeria de Telecomunicació de Barcelona

telecos BCN

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

FIB

Contents

Introduction	1
Design decisions	1
Code repositories visited	5
Bibliography	6

Introduction

We will be creating a web application for a band… Our band, SNAP!

Our application is a private web platform that record labels will use to store and share content with its bands and with the people who work around them: crew and management, press and business partners, and fans. The content includes unreleased recordings (demos, stems and masters), lyrics, setlists, tour riders, contracts, photos, videos and press kits.

Today this kind of material is usually shared by email. However, email cannot revoke access once a file has been sent, cannot enforce that some album sent to journalists can't be leaked before the official release, and leave no record of who accessed a file. Our app will solve that by granting access per user, per role, per band and per time window, so that each person only sees what they are allowed to when they are allowed to see it.

Design decisions

Users and roles

There are two types of accounts, professional accounts (band members, crew and press) and fan accounts. The difference is that professional accounts are created by an invitation of the admin, and fans can register themselves.

Role	**People**	**Permissions**
Administrator	Label staff	Can create bands, invite users, assign roles and verify accounts. Cannot open band content unless granted access.
Band Member	Musicians of a band	Can upload content of their band, edit and delete it.
Crew / Management	Tour managers, technicians	Access to schedules and tour info from the bands.
Press / partner	Journalists, producers, radio	Access music and material of the bands allowed to them, and after a certain date.
Fan club member	Subscribers of a band	Access music in pre-release, behind the scenes and early announcements.
Registered user	People with an account	Can access public content only.

Planned security features

We will choose four to six of the following security features in the table below:

Security Feature	**Plan**
Server Certificate (TLS)	We sign a certificate using our own authority, with OpenSSL, and use it with a nginx reverse proxy for HTTPS (this allows any user/admin in the internet to see the Authenticity of the server). Also, we should use session cookies marked as Secure, HttpOnly and SameSite.
Auth with OAuth 2.0	Fan accounts will log in with their Google Account using OpenID connect with Authlib. We are also considering Keycloak.
Access control with XACML 3.0	Use an AuthzForce server to see if the user is allowed to see the contents. For request, some kind of XACML request will be created, requesting subject, resource, action and environment attributes.
Password protection in the database	We will hash the professional accounts' passwords with Practically Secure algorithms (algorithms that would take hundreds of years to break) such as Argon2id, with unique salt. We will follow something like the OWASP Password Storage Cheat Sheet, and rate-limit login attempts and implement two factor auth, for instance with TOTP.
Content encryption at rest	Uploaded files encrypted with their own random key (AES-256-CGM). The key will be stored inside the database, wrapped with a master key kept in Docker Secrets.
Partial database encryption	Field-level AES-256-GCM encryption for sensitive columns (contract amounts, fees in tour riders, personal contact details) using a separate key from the one used in the files. Also, to make emails searchable, we will first compute a hash of the unencrypted email, then encrypt it, and finally store pairs (encrypted_email, hash) for lookup.

In addition, we would like to consider a feature: watermarking that depends on the recipient of some files (embedding the identity of the recipient in each downloaded file, so that a leak can be traced back to its source), and maybe embedding it too to the downloading links. Another idea is to use CI git automation to detect security flaws when pushing, such as exposed routes or private keys.

Functionality and pages

The application will provide all the functionality required by the practice: user login, access to restricted resources based on users and roles as described before, upload of new docs and resources and management of access permissions. It will also include the optional functionalities of user registration (for fans) and modification and deletion of existing resources. The planned pages are:

Page	**Description**	**Availability**
Login/Registration	Login with username and password (professional accounts) or with Google Account for fans. For privileged roles, we will require 2FA.	Everybody
Dashboard	Lists the content the user is allowed to see. We could group by band and category.	All authenticated Users
Content Detail	Displays item metadata with access-controlled options to view, stream, or download	Users with access to the item
Upload	Upload a file and set its details, such as band, category and download permission, and when the file will be available for the press and for fans.	Band Members
Permission Management	Change the audience level or the max date to see the file, and grant give or revoke access to some users.	Members of the owner Band
Access log	History of who viewed, streamed or downloaded each item, including denied attempts.	Band members (own band), administrators

Administration Panel	Management of bands, invitations, users and roles.	Administrators

When someone accesses the app, that person sees the dashboard. Then, depending on the permissions, they can open a detail of an item, and if they are band members then they can upload content or change permissions of current content. The administrator panel is apart and only for admins.

Architecture and technologies

Component	**Technology**	**Goal**
Reverse proxy	Nginx	For TLS termination and for security headers
Web app	Python (Flask)	For the user interface, auth and encryption and decryption of content
Database	PostgreSQL	Band contents, wrapped file keys and grants and access log
Choosing to allow the file to the user or not	AuthzForce Server	Evaluate the policies regarding the allowance
File storage	Docker volume	Here we will have the encrypted files of the app
Secrets	Docker secrets	For the keys, db credentials and for the OAuth client secret

Code repositories visited

AuthzForce Server: XACML 3.0 PDP/PAP server with a REST API.
https://github.com/authzforce/server

AuthzForce Core: XACML 3.0 PDP engine used by AuthzForce Server.
https://github.com/authzforce/core

Flask: Python web framework for the application.

https://github.com/pallets/flask

Authlib: OAuth 2.0 and OpenID Connect client library for Python.
https://github.com/lepture/authlib

Keycloak: Open-source identity provider (under evaluation).
https://github.com/keycloak/keycloak

argon2-cffi: Argon2 password hashing for Python. https://github.com/hynek/argon2-cffi

pyca/cryptography: AES-GCM encryption and key wrapping for Python.

https://github.com/pyca/cryptography

pip-audit: Detection of known vulnerabilities in Python dependencies.
https://github.com/pypa/pip-audit

Bandit: Static analysis of Python code for security issues.

https://github.com/PyCQA/bandit

Semgrep: Rule-based static analysis.

https://github.com/semgrep/semgrep

Gitleaks: Detection of secrets committed to the repository.

https://github.com/gitleaks/gitleaks

Trivy: Vulnerability scanner for container images.

https://github.com/aquasecurity/trivy

OWASP ZAP: Dynamic application security testing (DAST).
https://github.com/zaproxy/zaproxy

Bibliography

[1] OASIS. eXtensible Access Control Markup Language (XACML) Version 3.0. https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[2] FIWARE. Administrating XACML Rules (AuthzForce tutorial). https://fiware-tutorials.readthedocs.io/en/latest/administrating-xacml.html

[3] IETF. RFC 6749: The OAuth 2.0 Authorization Framework. https://datatracker.ietf.org/doc/html/rfc6749

[4] IETF. RFC 7636: Proof Key for Code Exchange by OAuth Public Clients (PKCE). https://datatracker.ietf.org/doc/html/rfc7636

[5] OpenID Foundation. OpenID Connect Core 1.0. https://openid.net/specs/openid-connect-core-1_0.html

[6] IETF. RFC 9106: Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications.

https://datatracker.ietf.org/doc/html/rfc9106

[7] OWASP. Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[8] OWASP. Cryptographic Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

[9] NIST. SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. https://csrc.nist.gov/pubs/sp/800/38/d/final

[10] IETF. RFC 6797: HTTP Strict Transport Security (HSTS). https://datatracker.ietf.org/doc/html/rfc6797

[11] Mozilla. SSL Configuration Generator. https://ssl-config.mozilla.org/

[12] OWASP. OWASP Top 10. https://owasp.org/www-project-top-ten/